算分第4次作业

郭劲豪 2200013146@stu.pku.edu.cn

第四章

4.1

(1)

我们选择按照t从小到大排序，然后按照这个顺序来依次服务对象的顺序。

(2)

算法:

```
sort(t);
T = 0;              /** 统计当前消耗的时刻T*/
ans = 0;            /** 统计所有顾客的等待时间和*/
for i in range(1, n) :
    ans += T;
    T += t[i];
print(ans)
```

算法所需要的时间复杂度为$O(n\log n)$

(3)

我们将服务时间排序后得到：{1, 2, 3, 6, 10, 12, 15}。
然后模拟算法的进行过程，得到最后答案为：0 + 1 + 3 + 6 + 12 + 22 + 34 + 49 = 78 + 49 = 127

4.3

设计思想：我们从d_1开始遍历，定义当前覆盖范围为x,每访问到一个未被覆盖的房子，就在该房子向B的方向4km处修建一个基站，然后继续向后遍历，最后统计修建了多少个基站。

伪代码描述:

```
x = d[1] + 4
ans = 1
for i in range(1, n):
    if d[i] > x + 4:
        x = d[i] + 4
        ans++
        location[ans] = d[i] + d
```

正确性证明：我们考虑如果之前的最优基站序列为$location_1,\ldots,location_{k-1}$，现在我们选取$location_k$的位置，当前第一个不能够被覆盖的房间为$d_i$，那么：$d_i-4\le location_k\le d_i+4$，在能够覆盖的情况下我们肯定尽可能选取最靠近右边的最优，这样子就可以扩大更多的覆盖范围，所以$location_k=d_i+4$，也就是和我们的算法描述做法一致。

时间复杂度：$O(n)$

4.6

我们按照加工时间t从小到大排序，然后依次将他们加工即可，设这样子的调度序列为$p_1, p_2, \ldots, p_n$，那么考虑交换p_i和p_j，$i-1$之前的ω和j之后的$omega$都不会改变：

$$\begin{aligned}\omega_i' &= f_i + t_{p_j} \\ \omega_{i+1}' &= f_i + t_{p_j} + t_{p_{i+1}} \\ &\cdots \\ \omega_j' &= f_i + \sum_{k=i}^{j} t_{p_k}\end{aligned}$$

发现对这个求和，比对ω求和大了$(j-i) \times (t_{p_j} - t_{p_i})$，所以这个调度策略是最优的。

4.13

设计思想：按照m_i排序，将m最大的前D个任务完成，剩下的选择赔偿罚款。

正确性证明：因为D是常数，所以我们考虑在最优的情况下，一定是完成若干项工作，然后剩下的工作进行罚款。设罚款的为$p_1 < p_2 < \ldots < p_k$，那么很明显：

$m_1 \le m_{p_1}, m_2 \le m_{p_2}, \ldots, m_k \le m_{p_k}$.

所以对他们求和之后也有$\sum_{i=1}^{k} m_i \le \sum_{i=1}^{k} m_{p_i}$，即这是最优解。

时间复杂度： $O(n \log n)$

4.18

(1)

算法：本质上就是线段覆盖问题，而且d已经排好序了，所以考虑设x为当前的最右端，如果$s_j \le x$，那么就把x替换为d_j，然后答案累加即可。

正确性证明：数学归纳法。

考虑$k=1$，选取$p_1 = 1$即可。

若对于k成立，我们考虑将当前的最优解称作$p_1 = 1, p_2, \ldots, p_k$，那么总的最优解可以分解为$A = \{p_1 = 1, p_2, \ldots, p_k\} \cup B$，这个时候由于$p_{k+1}$实际上和前$k$个都不交，所以我们可以钦点$B_1' = p_{k+1}$，因为都是最优解，所以$|B'| = |B|$，此时最优解就可以进一步分解为$A = \{p_1 = 1, p_2, p_3, \ldots, p_k, p_k + 1\} \cup B''$，得证。

时间复杂度：$O(n)$

(2)

设f_i表示在时间i之前的最大总收益，考虑动态规划：

$f_i = \max\{f_{i-1}, f_{s_k} + v_k(i = t_k)\}$

离散化之后时间复杂度可以做到：$O(n \log n)$

4.20

设计思想：我们按照许可证指数因子从大到小排序，然后按照这个顺序依次购买。

正确性分析：考虑交换p_i和p_j（$i < j$），那么答案将会增加$(r_{p_j})^j - (r_{p_i})^j - ((r_{p_j})^i - (r_{p_i})^i)$，不妨设$a = r_{p_i}, b = r_{p_j}$，则：

$$
\begin{aligned}
b^j - a^j &= (b-a)(b^{j-1} + b^{j-2}a + \ldots + a^{j-1}) \\
b^i - a^i &= (b-a)(b^{i-1} + b^{i-2}a + \ldots + a^{i-1})
\end{aligned}
$$

由结合律易得，$b^j - a^j > b^i - a^i$，所以答案会增加，由交换论证可得，答案为最优解。

时间复杂度：$O(n \times k + n \log n)$